
Compiler-Guided Autoformalization with Adaptive Multi-View Representations

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Autoformalization must preserve an informal requirement’s meaning while pro-
2 ducing an artifact that a machine can check. We investigate a compiler-guided
3 architecture coupling typed, source-grounded intermediate representations with
4 learned reconstruction and isolated proof-feedback heads. We distinguish struc-
5 tural validity, source fidelity, and proof acceptance throughout its evaluation. Na-
6 tive compilers and an exception-scoping repair are evaluated on 1,913 held-out
7 legal-source units; direct model formalization uses a fixed 100-unit subset. Across
8 three training seeds, a learned projection attains mean embedding cosine 0.560,
9 compared with 0.196 for a fixed cold projection and 0.538 for a constant training-
10 mean control. The decoder receives the source embedding, so these results do not
11 establish compression or independent semantic fidelity. A learned prior changes
12 the order of required structural checks while preserving their outcomes. On 64
13 constructed proof goals, Leanstrat supplies 12 kernel-accepted proofs among 32
14 valid goals and none among 32 invalid controls; a fixed native tactic baseline
15 accepts all 32 valid goals. The study identifies operational behavior and failure
16 modes within the architecture. Independent legal-semantic fidelity and general
17 natural-source proof transfer remain unmeasured.

18 1 Introduction

19 A well-typed formula can misrepresent its source, and a correct proof can establish the wrong re-
20 quirement. This distinction matters when autoformalization extends beyond mathematical state-
21 ments to policies, programs, and execution traces. An obligation is not an observation, permission
22 is not execution, and belief is not fact. Mathematical autoformalization has made natural-language
23 translation into proof-assistant statements increasingly practical [Wu et al., 2022, Azerbayev et al.,
24 2023, Jiang et al., 2024]. For verifiable coding, the corresponding question is how to improve
25 source-to-symbolic mappings while preserving these distinctions and retaining an explicit basis for
26 accepting a result.

27 We investigate a compiler-guided architecture. Deterministic compilers produce typed intermediate
28 representations (IRs) with source spans and modal profiles. Decomposers expose logical views;
29 decompilers reconstruct inspectable content; learned components model reconstruction and propose
30 priorities. Feedback can update shared parameters, train isolated prediction heads, or motivate an
31 executable compiler repair. Structural checks and proof kernels retain authority over their respective
32 acceptance decisions. The schema, operators, sorts, and admissibility rules are engineered; learning
33 operates within those contracts.

34 We make three contributions:

- 35 1. We specify separate contracts for source-grounded compilation, source-withheld recon-
36 struction, and conditional proof transfer between logical views.

2. We evaluate native compilation and a paired exception-scoping repair on 1,913 held-out source units, with a direct-model comparison on a prospectively selected 100-unit subset. We report representation checks, reconstruction behavior, abstention, and runtime failures without interpreting them as legal-semantic accuracy.
3. We evaluate actual three-seed learned states through a source-conditioned inference boundary without target-guided selection, exercise a learned check-order consumer with rollback, and compare native proof assistance on 64 constructed valid and invalid goals.

This systems study evaluates the implemented components without treating a compiler or prover as a surrogate for independent interpretation of a legal source. End-to-end semantic improvement remains unmeasured.

2 Representations, teachers, and reconstruction

2.1 What is shared, and what is learned

Three axes remain distinct. Source format names prose, code, traces, tables, or extracted media. Domain names Legal, Security, Intent, software, or UI declarations. Logical view names deontic, temporal, frame, first-order, cognitive/event, graph, or proof-oriented structure. Sharing canonical records does not equate domain ontologies or logical interpretations.

The domain-neutral kernel supplies canonical encodings, identities, provenance, source spans, assumptions, claims, and result protocols; domain adapters retain their semantic slots. A content identifier binds a discrete encoding under a declared profile. Canonical declarations, logical formulas, numerical features, graph relations, search plans, and proof records retain separate roles: vector proximity does not imply formula equivalence, and a search plan does not establish its goal.

2.2 Supervisory targets are teachers, not source gold

Legal training records bind source text to an identified embedding producer, modal IR, frame candidates, parser trace, and losses. Modal-family targets come from parsed operators; view targets come from bridge contracts; frames may be retrieval-selected; proof-feedback labels require admitted verifier records. Reproducing these labels measures teacher distillation. Fidelity to the original source requires a separate evaluation, which our automated targets do not supply.

For a source x , write $I = C_\psi(x)$ for the symbolic compiler output, $e = T(x)$ for the declared embedding target, and $f = F(x, I)$ for reusable features. Features include lexical cues, actor/action/object roles, conditions and exceptions, temporal structure, predicate arity, quantifier scope, provenance, frame relations, compiler contracts, and reconstruction diagnostics. The adaptive model predicts an embedding $\hat{e}$, a modal-family distribution p_F , and a view distribution p_V . Some features depend on an existing parse, so this is often parser-assisted learning, not a demonstrated raw-text-only compiler.

2.3 Compilers and two meanings of reconstruction

The local modal codec uses deterministic registry cues, spaCy-derived features, BM25 frame grounding, and frame-logic consistency checks. A selected structured-text profile has a seven-field rule—modality, actor, action, object, conditions, exceptions, and temporal qualifiers—and explicitly disables learned guidance and repair. Formalization adapters produce named formula views and source maps. The maps bind IR constituents to source evidence; they are not proof of a correct interpretation. Media extraction is a limitation: no OCR/ASR theorem performance is claimed.

Symbolic reconstruction follows

$$I_1 = C_\psi(x), \quad \hat{x} = D_\omega(I_1), \quad I_2 = C_\psi(\hat{x}). \quad (1)$$

A decompiler renders roles, predicates, modality, conditions, exceptions, and temporal structure into inspectable text. Source-withheld evaluation gives the realizer the IR, not the originating source, locator, source map, private parser state, or gold target. Comparing I_1 and I_2 tests reconstructable facets and compiler/realizer consistency. It does not prove source fidelity: both programs could

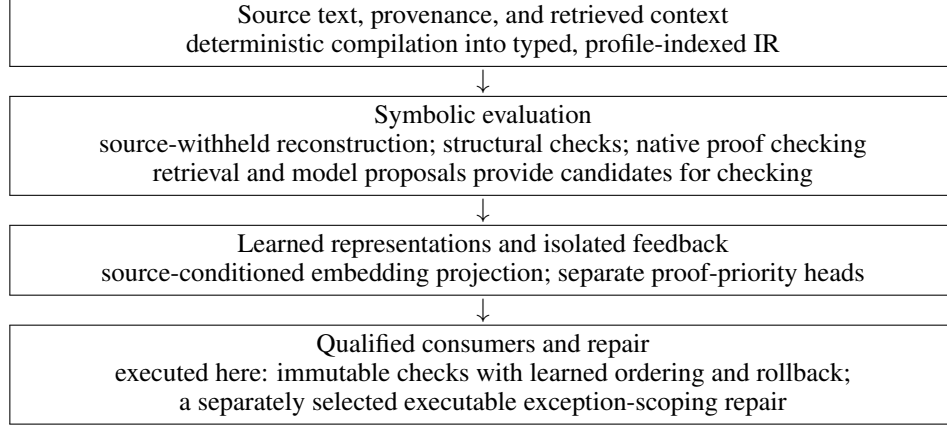


Figure 1: Architectural overview. Information-flow contracts connect compilation, reconstruction, assistance, and learning. The experiments exercise these components separately; the diagram does not imply an automatically activated end-to-end source-to-proof system.

consistently omit the same exception. Forward (gold→first IR), cycle (first IR→second IR), and end-to-end (gold→second IR) scores remain distinct.

Numerical reconstruction has a different target: encode builds family predictions and an embedding projection, and decode returns a vector. This does not implement a trained free-text decoder. The evaluated consumer excludes the legacy target-guided shortcut described below.

3 Adaptive learning and assistance

3.1 Shared parameters versus sample memory

The adaptive implementation has sparse parameter groups for feature embeddings, modal families, semantic slots, predicate arguments, logic signatures, view logits, decompiler plans, and family-slot-view interactions. It also retains compatibility fields indexed by sample identity. The generalizable training route disables per-sample memory for updates and evaluation. Reconstructing a stored vector for a known sample is not generalization. A schematic of the shared heads is

$$\hat{e} = e_0(e, f) + \sum_g A_g(f)W_g, \quad (2)$$

$$p_F = \text{softmax}\left(\ell_F^0(f) + \sum_g A_g(f)U_g\right), \quad p_V = \text{softmax}\left(\ell_V^0(f) + \sum_g A_g(f)V_g\right). \quad (3)$$

Here A_g is activity of a bounded feature group, and W_g, U_g, V_g are enabled embedding and logit tables. This notation summarizes the packed implementation; it is not a Transformer or variational autoencoder. A forward-only no-op is reported separately from a parameter update.

3.2 Packed objective and isolated proof heads

The packed tensor path gathers active rows, reduces losses in FP32, and applies a clipped SGD-style update to transaction-aware state. For the active heads,

$$L_{\text{packed}} = H(q_F, p_F) + H(q_V, p_V) + \text{MSE}(e, \hat{e}) + \lambda_c(1 - \cos(e, \hat{e})) + L_{\text{norm}} + \lambda_2 R(\Theta). \quad (4)$$

Targets are normalized distributions; masks exclude unavailable rows. Neither cosine similarity nor cross-entropy is a theorem about recovered meaning. Appendix B records the globally normalized masked CE and the SGD-style step actually audited.

Seven explicit heads can predict obligation family, semantic slots, premise family, route availability, trusted outcome, reconstruction success, and minimal failing contract. Their declared weight in the

primary representation objective is zero. A prediction of proof success can prioritize a route but cannot certify a formula.

The retained T2 checkpoints contain changed embedding-projection tables but empty modality and view-logit tables, so they establish no learned classification gain. T3 preserves the reconstruction parameters and adds an obligation-family prior; its other single-label heads provide no calibrated correctness estimate. Our guidance consumer uses this prior to order two required checks. Both checks run on the unchanged IR under fixed contracts, so ordering cannot increase coverage by skipping obligations.

3.3 Retrieval, planning, and proof proposals

BM25, vector indexes, and graph traversal retrieve complementary context. Context becomes a premise only after grounding and applicability checks; retrieval failure is not logical negation.

The Tactician orders provided sources and constructs an acyclic subgoal plan. The ITP Hammer lowers a supported first-order fragment, invokes a solver portfolio, and attempts reconstruction in the originating checker. Leanstral likewise supplies advisory proposals. Acceptance requires native checking and excludes `sorry`, `admit`, and unapproved axioms. Our assistance study exercises these boundaries on constructed goals with explicit premises using Lean 4.33.1, Z3 4.15.4, CVC5 1.3.3, and local Leanstral-1.5.

Inference boundary. A legacy helper could select a reconstruction against the supplied target, including returning the target itself. Our consumer bypasses it, fixes its alphabet from training-checkpoint keys, and writes a direct projection before scoring, without retraining. The source embedding remains an input: identity copying would attain cosine 1 and MSE 0. The experiment therefore measures a source-conditioned projection, with neither compression nor independent textual meaning established.

4 Multi-view translation and a coding example

4.1 Profile-indexed semantics and a conditional transfer rule

Each logical view has a signature, sentence language, semantic models, satisfaction relation, and a supported query fragment. For illustration, a normal modality can be interpreted with an indexed accessibility relation

$$\llbracket \Box_i \phi \rrbracket(w) = \forall v [R_i(w, v) \Rightarrow \llbracket \phi \rrbracket(v)]. \quad (5)$$

Equation 5 is an illustrative semantic recipe, not a claim that every repository view is compiled by it. Ordinary normal modal logic is not a universal deontic solution [Benzmüller et al., 2020]. $O(Gp)$ and $G(Op)$ are not equated without a profile-specific commutation result.

A translation record must state source/target views, profile, signature map, assumptions, supported constructs, named losses, preserved property, and checker evidence. Translation validation examines the produced transformation rather than trusting every compiler invocation [Pnueli et al., 1998]. A legacy TDFOL-to-FOL converter that erases unary temporal and deontic operators and replaces strong until by conjunction is a negative control; a resulting solver success is not a valid modal proof.

Let $\Gamma \models_{L_s} \phi$ be the desired source-view consequence. A translator τ maps premises and goal into L_t . Under bridge assumptions B , let $R_B(s, t)$ relate admitted source and target models. The required directionality is

$$s \models \Gamma \wedge R_B(s, t) \Rightarrow t \models \tau(\Gamma), \quad (6)$$

$$t \models \tau(\phi) \wedge R_B(s, t) \Rightarrow s \models \phi. \quad (7)$$

For every admitted source model satisfying Γ and B there must be a related target model; otherwise the relation can be vacuous. If a target checker soundly establishes $\tau(\Gamma) \models \tau(\phi)$, these conditions transfer the consequence back. This is a *conditional paper proposition* to discharge per supported translation, not a machine-checked soundness theorem about every implementation translator. The transfer contract also requires consistent premises; explosion cannot justify a useful transfer claim.

153 4.2 Synthetic policy–code–trace illustration

154 Consider a synthetic coding requirement: “Protected resources must not be written without approval.
155 Each write must be audited within two steps.” It is a specification example, not legal advice and not
156 a production experiment. One source supplies the requirement, a second supplies implementation
157 text, and a third supplies observations. The normative view may represent the first sentence as

$$\text{Protected}(r) \wedge \neg \text{Approved}(a, r, t) \Rightarrow O_{a,t,\theta}(\neg \text{Write}(a, r, t)). \quad (8)$$

158 The policy-to-compliance bridge defines which observed behaviors count as violations. It does not
159 derive actual non-writing from the obligation. The compliance queries are

$$Q_1 = \forall a, r, t : \text{Protected}(r) \wedge \neg \text{Approved}(a, r, t) \Rightarrow \neg \text{Write}(a, r, t), \quad (9)$$

$$Q_2 = \forall a, r, t : \text{Write}(a, r, t) \Rightarrow \exists u \in [t, t + 2] : \text{Audit}(a, r, u). \quad (10)$$

160
161 A deterministic implementation model $\text{Write} = \neg \text{Protected} \vee \text{Approved}$ makes Q_1 hold on that
162 finite Boolean model. An always-write implementation supplies a countermodel when the resource
163 is protected and approval is false. Transferring either result to actual source code requires a justified
164 extraction or refinement relation; finding the word “approve” in a function name is insufficient. For
165 Q_2 , a write at step 0 and a matching audit at step 1 satisfy the bounded requirement for that observed
166 event; an audit at step 3 violates it only when capture is complete through the deadline; an incomplete
167 window yields unknown; a same-looking name in a different tenant cannot be joined without an
168 admitted identity mapping. These are finite constructed witnesses, not compiler soundness and not
169 Table 1.

170 5 Experimental design

171 5.1 Data, separation, and budgets

172 The frozen legal-source cohort contains 69 training units, 15 selection units, 38 fixed canaries, and
173 1,913 final-test units. The latter comprise 993 CFR, 738 MPEP, 175 U.S. Code, and seven guid-
174 ance units. Eight exact-normalized aliases are retained as provenance but excluded from statistical
175 denominators. Components were constructed before model evaluation from source-family, edition,
176 and lexical/paraphrase relationships. The final set contains 20 operational components across four
177 source editions, with one component containing 1,738 raw records (1,730 distinct units). These are
178 not 20 independent legal corpora. All training and selection units are CFR; the final cohort therefore
179 includes substantial domain shift.

180 We froze the primary inference code, checkpoint identities, compiler profiles, prompts, sampling,
181 and metric definitions before opening final-test sources. Compiler inputs use the first 4,096 char-
182 acters of each source; we report the resulting truncation count. Embeddings use the complete doc-
183 ument under the original MiniLM windowing contract. The direct-model subset is the 100 units
184 with the lowest salted-prefix hash of normalized source content. Its prompt contains source text and
185 the target schema, with no compiler output or evaluation label. The subset comparison reuses the
186 matching compiler rows; it is not a model result over all 1,913 units.

187 Compiler B has a five-second per-unit budget. C and its repaired counterpart have ten seconds,
188 including their reconstruction cycle. For C/T5, structural checks finish before cycle evaluation,
189 so a later cycle timeout retains the compilation result. B’s native round-trip call returns a single
190 bounded result and does not preserve partial compilation on timeout; its reported success therefore
191 has a different observation boundary. The temporary Leanstral jobs use a 90 GiB memory limit
192 and four CPU cores. Proof jobs have a 900-second wall envelope. The natural-source job uses
193 a 10,000-second envelope, keeping its model loaded across client batches until all requests finish.
194 Each request uses temperature zero, one attempt, and a 60-second timeout. Proof requests allow
195 1,024 output tokens; direct formalization permits 2,048 tokens and at most eight formulas, followed
196 by a separately bounded ten-second reconstruction cycle. Costs include failed startup qualification
197 and completed attempts. Concurrent execution makes observed latency descriptive, rather than a
198 controlled throughput comparison.

199 5.2 Native compilation and reconstruction

200 We distinguish the architectural pipeline labels A–E from the learning labels T0–T5. A is direct
201 model formalization; B is a bounded typed deontic compiler; C uses typed multi-view IR; D adds

property-specific checked bridges; E admits learned advice. The natural-source experiment executes A on 100 units and B/C on the complete cohort. A separate constructed study tests proof-assistance machinery relevant to D. A separately qualified learned check-order consumer tests one component relevant to E; it does not execute the broader semantic-mutation or complete source-to-proof pipeline.

B invokes the native compiler and native semantic round-trip implementation with a deterministic source-derived vocabulary, capped at 1,000 atoms per category, and strict abstention on unsupported input. C invokes the native spaCy legal encoder, ModalIR compiler, and clause enrichment. It omits the full codec’s knowledge-graph and embedding decoration, which are unnecessary for these representation checks. The recorded spaCy model configuration is used without fitting on the final set.

For A, C, and T5, two software checks test registered modal operators and typed formula fields, and bind normalized source content, identifiers, and span bounds. An empty, well-shaped IR may pass the schema; it does not count as nonempty coverage. These checks establish neither predicate truth nor legal interpretation. For C/T5 and accepted A outputs, we remove the original text, metadata, source identifiers, spans, and frame fields before native decompilation. Operator, predicate, condition, and exception fields remain because they are the representation being evaluated. Recompile uses the same bounded source profile. Their cycle metric compares semantic-field formula multisets and excludes empty-to-empty matches. B instead validates its native CID evidence chain and compares the complete canonical rule IR across its source-withheld round trip.

T5 applies the previously developed executable exception-scoping patch to C. It separates exception predicates from governing actions and labels exception cues explicitly. The held-out comparison measures changed fields and structural regressions; it does not supply independent semantic labels for those changes.

5.3 Learning and proof assistance

T0 is the native cold projection: $0.02e + 0.10Je$, where J rotates each coordinate pair by a quarter turn. It is a fixed imperfect fallback, with cosine $0.02/\sqrt{0.02^2 + 0.10^2}$ for a nonzero even-dimensional input, rather than a competitive representation baseline. T1 adds the retained 69-entry sample memory. T2 contains shared parameters trained with memory disabled; T3 adds isolated structural-feedback heads. The three fixed seeds are 104729, 130363, and 155921. Each retained T2 run accepted one epoch with five packed update families; each T3 run applied 256 structural-feedback updates. We verified the original state hashes and extracted exact compact inference arrays. MiniLM re-encoding matched all 15 retained selection vectors bit for bit. T3 reconstruction values are reused from T2 only after exact equality of protected decoder state is checked; they are not an independent replication. T1 performs actual native memory lookups, with verified cold fallback on misses.

A supplemental control predicts the L2-normalized mean of the 69 training vectors for every source. We fixed this control after final-source release but before any final autoencoder predictions or scores, using training vectors alone. It contextualizes the original primary T2–T0 comparison without replacing it. An execution-only cache and worker partition were separately qualified for exact equivalence on all 15 selection units before final projection inference; neither changes learned parameters or the frozen prediction function.

T4 evaluates the qualified learned priority through off/on/rollback execution on each source and seed. Every mode runs both checks. The priority is a learned global frequency prior, not a source-adaptive estimate of proof success. We measure activation, order, identical acceptance outcomes, and restoration of the baseline after rollback.

The proof study contains eight families, each with four valid and four invalid constructed goals: premise transfer, polarity, equality transport, integer identities, natural order, finite next-state preservation, dependent equality, and higher-order propositions. Native controls validate the labels, including explicit countermodels for invalid higher-order and dependent families. A fixed tactic baseline, SMT-guided reconstruction, and Leanstrat receive identical theorem declarations. SMT statuses alone are never accepted as Lean proofs. Generated proof bodies must pass a fixed syntax policy and Lean kernel checking, with no added axioms or unfinished proof terms. The standard Lean

Table 1: Native representation and reconstruction outcomes. A and C share structural checks on the 100-source subset. B uses its stricter native deontic contract; its whole-call timeouts cannot be assigned to a cycle phase. Counts are not source-semantic accuracy.

Method	Units	IR passes	Nonempty	Exact cycle	Cycle timeout
Native B	1913	32	32	26	—
Native C	1913	1913	1893	0	809
Repaired C (T5)	1913	1913	1893	0	911
Direct model A	100	62	53	0	0
Native C, same subset	100	100	99	0	44

axioms `propext`, `Classical.choice`, and `Quot.sound` are permitted and reported. Five older development goals qualify serving but do not enter the 64-goal denominator.

Statistics. We report all planned denominators, separating abstention, timeout, malformed output, empty representation, and successful checks. Paired differences use 10,000 bootstrap resamples of the original source components with seed 104729. Domain-stratified unit resamples are secondary, conditional descriptions of this finite cohort. The dominant component and four-edition dependence limit generalization of either interval. Proof-study intervals resample the eight constructed families; these template variants are not independent real-world theorems.

6 Results

6.1 Compilation and source-withheld reconstruction

Native C produces 1,893 nonempty checked representations, including 1,708 units containing a deontic formula. Its 26,784 formulas span the registered logical families. Nonempty coverage alone cannot distinguish useful formalization from spurious output. Only 0 units reproduce their semantic-field multiset exactly; 809 cycle attempts time out. A completed initial compilation is retained when a later cycle times out. The source-prefix limit truncates 971 of 1,913 inputs.

B records 32 successful initial compilations. Its full pipeline returns 31 successes, 479 abstentions, 199 native failures, and 1,204 whole-call timeouts. Its strict vocabulary and abstention contract differ from C’s richer output contract, so these counts are operational coverage under different interfaces, not a ranking of semantic accuracy.

On the matched 100-unit subset, A yields 71 accepted-format responses, 62 passing both checks, and 53 nonempty checked IRs; 9 passing IRs are empty. There are 26 format rejections and 3 request timeouts. C yields 99 nonempty checked IRs on those same sources. Structural passage remains weaker than faithful interpretation. In one retained A output, the source’s permission that the USPTO may disclose a record becomes an obligation, and the cited span covers a heading. This post-hoc example illustrates a check’s limitation; it is not an annotated error-rate estimate.

6.2 Reconstruction, learned priorities, and repair

The seed-averaged T2–T0 cosine difference is +0.3638 (component-bootstrap 95% interval [+0.3519, +0.4164]); the MSE difference is -0.000704 ([-0.000838, -0.000668]). The constant training-mean cosine falls below the learned projection’s seed average (0.5381 versus 0.5599). The range of mean cosine across the three stored seeds is 1.1×10^{-9} . The cold-baseline difference alone therefore does not establish a strong representation-learning result. T1 records 0 memory hits in 5,739 native lookups. Its unseen-source scores equal T0 exactly. Protected-state equality establishes decoder isolation; reused T3 scores do not measure an additional gain from proof feedback.

Across 5,739 source–seed pairs, T4 consumes learned state 5,739 times and changes check order in 5,739 pairs. Outcomes remain unchanged in 5,739 pairs, with 5,739 successful rollbacks and 34,434 actual check calls. This verifies consumption of the learned prior and preserves the acceptance contract; it does not demonstrate better semantic interpretation or fewer required checks.

Table 2: Source-conditioned embedding reconstruction on 1,913 held-out units. T3 reuses exactly tied decoder values. The decoder has no demonstrated compression bottleneck; identity copying is exact by construction. The supplemental constant mean uses only 69 training vectors.

Condition	Seed	Mean cosine	Mean MSE
T0 fixed cold projection	fixed	0.1961	0.002527
T1 sample memory	three seeds	0.1961	0.002527
T2 shared parameters	104729	0.5599	0.001823
T2 shared parameters	130363	0.5599	0.001823
T2 shared parameters	155921	0.5599	0.001823
T3 isolated proof heads	tied to T2	same as T2	same as T2
Training-mean control	supplemental	0.5381	0.002406
Identity-copy control	analytic	1.0000	0.000000

Table 3: Native acceptance on 64 constructed goals (32 valid, 32 invalid). Every method attempted the same fixed declarations.

Method	Valid accepted	Invalid accepted	Unsupported
Fixed native tactics	32/32	0/32	0
SMT-guided reconstruction	24/32	0/32	16
Leanstrat, primary policy	12/32	0/32	0

T5 changes canonical semantic fields on 619/1,913 units (component-bootstrap 95% interval for the changed fraction [23.6%, 33.0%]). Within pairs with equal formula counts, it changes predicate fields outside the conditional-normative family on 403 units; it increases explicit exception roles on 618. There are 0 regressions in the frozen schema/provenance checks. Cycle timeouts are 911 with the patch and 809 without it under concurrent execution; the patch does not improve exact-cycle coverage. These paired effects show execution of the patch; independent judgments would be needed to quantify whether each changed interpretation is correct.

6.3 Native proof assistance

Leanstrat’s valid-goal acceptance differs from the baseline by -62.5 percentage points (2,000 family-bootstrap draws; 95% interval [-90.6, -28.1]). The fixed baseline accepts all 32 valid controls, while SMT-guided native reconstruction accepts 24; the dependent and higher-order families fall outside the translator’s supported fragment. Leanstrat accepts 12 valid goals and no invalid control under the primary policy. Its remaining outcomes comprise 30 abstentions, 15 policy rejections, five native rejections, and two length-truncated responses. The totals refer to all 64 cases, including invalid controls. Thus the model does not improve on the baseline for these elementary goals.

A post-hoc interface sensitivity check admits two additional harmless syntax forms without editing or regenerating proof bodies. It adds three accepted valid proofs, giving 15/32 in that diagnostic. The primary result remains 12/32. This distinction matters because an admission-policy rejection is not necessarily a logical error. The local model also required an explicit tokenizer chat template: the GGUF lacked one, and the server’s default format was inappropriate. We qualified the corrected framing and special-token sequence before the new proof requests.

7 Related work

Mathematical autoformalization studies translation into proof-assistant statements [Wu et al., 2022, Azerbayev et al., 2023, Jiang et al., 2024]. Our legal-source experiment instead combines domain-specific parsing, structured teachers, and a bounded learned consumer. Its structural metrics are not directly comparable to mathematical statement-accuracy benchmarks. Retrieval-augmented and informal-to-formal proving [Jiang et al., 2022, 2023, Yang et al., 2023, First et al., 2023] connect language models to proof search. Thor and Sledgehammer [Blanchette et al., 2013] emphasize reconstruction of external solver candidates in the originating ITP; our constructed experiment tests that acceptance boundary.

322 CompCert [Leroy, 2009], seL4 [Klein et al., 2009], and Dafny [Leino, 2010] illustrate the connection
323 between specifications, implementations, and machine-checked artifacts. Our finite policy–code–
324 trace example motivates source extraction for that setting, without claiming a verified compiler
325 or operating-system kernel. Neural-symbolic VQA [Yi et al., 2018] separates learned perception
326 from symbolic execution. Institutions [Goguen and Burstall, 1992], LogiKEy [Benzmüller et al.,
327 2020], frame logic [Kifer et al., 1995], event calculus [Kowalski and Sergot, 1986], and translation
328 validation [Pnueli et al., 1998] provide established foundations for our profile-indexed account of
329 representation and transfer. The present experiments evaluate the implementation described here;
330 they do not re-benchmark these systems.

331 8 Limitations and conclusion

332 The measurements support a bounded empirical conclusion: native compilation, learned projection,
333 executable repair, and model-assisted proof checking can be evaluated separately within a shared
334 architecture. They also identify the limits of the current implementation. A source span identifies
335 a cited interval, not whether it supports the formula. Exact reconstruction can preserve an incorrect
336 interpretation. The embedding experiment has no compression bottleneck, its classification tables
337 are empty, and its proof-feedback heads provide a global prior rather than source-adaptive confi-
338 dence. The learned consumer schedules checks without changing which obligations must pass. The
339 elementary proof study favors a simple native baseline.

340 Independent legal-semantic fidelity, full-document compiler coverage, and useful cross-view proof
341 transfer from arbitrary natural sources remain open. The four-edition corpus, imbalanced compo-
342 nents, source prefixes, and constructed proof templates restrict external validity. Some MPEP text
343 includes navigation boilerplate; a bounded prefix can retain navigation while omitting substantive
344 content. Our contribution is the implemented coupling of these mechanisms and a reproducible
345 measurement of their actual behavior, including failures. Improving source interpretation will re-
346 quire explicit semantic targets and richer property-specific bridges, while keeping reconstruction
347 diagnostics and proof acceptance distinct.

References

- Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023. doi: 10.48550/arXiv.2302.12433. URL <https://arxiv.org/abs/2302.12433>.
- Christoph Benz Müller, Xavier Parent, and Leendert van der Torre. Designing normative theories for ethical and legal reasoning: LogiKey framework, methodology, and tool support. *Artificial Intelligence*, 287:103348, 2020. doi: 10.1016/j.artint.2020.103348.
- Jasmin Christian Blanchette, Sascha Böhme, and Lawrence C. Paulson. Extending Sledgehammer with SMT solvers. *Journal of Automated Reasoning*, 51(1):109–128, 2013. doi: 10.1007/s10817-013-9278-5.
- Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1229–1241, 2023. doi: 10.1145/3611643.3616243.
- Frosty40. Leanstral-1.5-119B-A6B-GGUF-NVFP4. Hugging Face model repository, 2026. URL <https://huggingface.co/Frosty40/Leanstral-1.5-119B-A6B-GGUF-NVFP4/tree/abcc5ce2528c6375148d41dac6dce20f06c339f4>. Apache License 2.0.
- Joseph A. Goguen and Rod M. Burstall. Institutions: Abstract model theory for specification and programming. *Journal of the ACM*, 39(1):95–146, 1992. doi: 10.1145/147508.147524.
- Albert Q. Jiang, Wenda Li, and Mateja Jamnik. Multi-language diversity benefits autoformalization. In *Advances in Neural Information Processing Systems*, volume 37, pages 83600–83626, 2024. doi: 10.52202/079017-2659. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/984de836e696ba653bbfbbbfce31d3bc-Abstract-Conference.html.
- Albert Qiaochu Jiang, Wenda Li, Szymon Tworowski, Konrad Czechowski, Tomasz Odrzygóźdź, Piotr Miłoś, Yuhuai Wu, and Mateja Jamnik. Thor: Wielding hammers to integrate language models and automated theorem provers. In *Advances in Neural Information Processing Systems*, volume 35, pages 8360–8373, 2022. doi: 10.52202/068431-0608. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/377c25312668e48f2e531e2f2c422483-Abstract-Conference.html.
- Albert Qiaochu Jiang, Sean Welleck, Jin Peng Zhou, Timothée Lacroix, Jiacheng Liu, Wenda Li, Mateja Jamnik, Guillaume Lample, and Yuhuai Wu. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=SMA9EAovKMC>.
- Michael Kifer, Georg Lausen, and James Wu. Logical foundations of object-oriented and frame-based languages. *Journal of the ACM*, 42(4):741–843, 1995. doi: 10.1145/210332.210335.
- Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, pages 207–220, 2009. doi: 10.1145/1629575.1629596.
- Robert A. Kowalski and Marek J. Sergot. A logic-based calculus of events. *New Generation Computing*, 4(1):67–95, 1986. doi: 10.1007/BF03037383.
- K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning*, volume 6355 of *LNCS*, pages 348–370, 2010. doi: 10.1007/978-3-642-17511-4_20.
- Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi: 10.1145/1538788.1538814.

396 Mistral AI. Leanstral 1.5 119B A6B. Hugging Face model repository, 2026.
 397 URL [https://huggingface.co/mistralai/Leanstral-1.5-119B-A6B/tree/](https://huggingface.co/mistralai/Leanstral-1.5-119B-A6B/tree/3fe4e64d70e6873420fe19108a3a61ec6a9f1460)
 398 [3fe4e64d70e6873420fe19108a3a61ec6a9f1460](https://huggingface.co/mistralai/Leanstral-1.5-119B-A6B/tree/3fe4e64d70e6873420fe19108a3a61ec6a9f1460). Apache License 2.0.

399 Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms*
 400 *for the Construction and Analysis of Systems*, volume 1384 of *LNCS*, pages 151–166, 1998. doi:
 401 10.1007/BFb0054170.

402 Sentence Transformers. all-MiniLM-L6-v2: Model snapshot. Hugging Face model repository,
 403 2026. URL [https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/](https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/tree/1110a243fdf4706b3f48f1d95db1a4f5529b4d41)
 404 [tree/1110a243fdf4706b3f48f1d95db1a4f5529b4d41](https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/tree/1110a243fdf4706b3f48f1d95db1a4f5529b4d41). Apache License 2.0.

405 Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jam-
 406 nik, and Christian Szegedy. Autoformalization with large language models. In *Advances*
 407 *in Neural Information Processing Systems*, volume 35, pages 32353–32368, 2022. doi:
 408 10.52202/068431-2344. URL [https://proceedings.neurips.cc/paper_files/paper/](https://proceedings.neurips.cc/paper_files/paper/2022/hash/d0c6bc641a56bebee9d985b937307367-Abstract-Conference.html)
 409 [2022/hash/d0c6bc641a56bebee9d985b937307367-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2022/hash/d0c6bc641a56bebee9d985b937307367-Abstract-Conference.html).

410 Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad
 411 Godil, Ryan J. Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with
 412 retrieval-augmented language models. In *Advances in Neural Information Process-*
 413 *ing Systems*, volume 36, pages 21573–21612, 2023. doi: 10.52202/075280-0944.
 414 URL [https://proceedings.neurips.cc/paper_files/paper/2023/hash/](https://proceedings.neurips.cc/paper_files/paper/2023/hash/4441469427094f8873d0fecb0c4e1cee-Abstract-Datasets_and_Benchmarks.html)
 415 [4441469427094f8873d0fecb0c4e1cee-Abstract-Datasets_and_Benchmarks.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/4441469427094f8873d0fecb0c4e1cee-Abstract-Datasets_and_Benchmarks.html).

416 Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Joshua B.
 417 Tenenbaum. Neural-symbolic VQA: Disentangling reasoning from vision and lan-
 418 guage understanding. In *Advances in Neural Information Processing Systems*, vol-
 419 ume 31, 2018. URL [https://proceedings.neurips.cc/paper/2018/hash/](https://proceedings.neurips.cc/paper/2018/hash/5e388103a391daabe3de1d76a6739ccd-Abstract.html)
 420 [5e388103a391daabe3de1d76a6739ccd-Abstract.html](https://proceedings.neurips.cc/paper/2018/hash/5e388103a391daabe3de1d76a6739ccd-Abstract.html).

A Actual training and promotion execution

The completed common-profile run used 2 CPU cores, 64 GiB RAM, 64 processes, no network, Torch intra-op threads 2/inter-op 1, and three unchanged seeds. T2’s native soft budget was 9000 s; an independent host limit enclosed its full 10800 s stage, including evaluation and reload. T1 and T3 were outside that stage but inside the 36000 s whole-run limit. This prospectively superseded the historical 16 GiB/1800 s and residual 600 s profiles; it did not refund prior failed usage.

Table 4: Actual exclusive training-stage wall seconds. All seeds accepted one T2 epoch with five packed update families; T3 applied 256 structural-feedback updates each. No final-test outcome is implied.

Seed	T1	Complete T2	Complete T3
104729	24.724	2628.314	1809.538
130363	25.368	2646.454	1812.073
155921	24.906	2693.777	1841.133

The retained training stages total 13,926.774 wall seconds, including common preparation and feedback. A subsequent checker exhausted memory after 764.651 seconds; checking the retained artifacts separately took 1,265.157 wall seconds and 1,258.410 process-CPU seconds. Earlier failed attempts and their original receipts are retained. These stage intervals overlap enclosing run boundaries, so they are not summed with whole-run elapsed times.

The historical semantic-mutation consumer was evaluated on 38 fixed canaries for each of three seeds. All 114 diagnostics completed, but no promotion admitted a candidate: proof receipts, view-family weights, resolved canonical contracts, or qualifying guidance records were absent. No learned semantic mutation was applied. The diagnostics used 1,380.308 wall seconds and 1,363.292 process-CPU seconds. The current T4 experiment evaluates a separately qualified check-order consumer; its activation does not retroactively establish that the earlier mutation route worked.

Training feedback consists of 69 source-bound packets: 61 passed structural checking, six timed out, and two were unsupported. Their 2,606 eligible labels describe compiler contracts. Each T3 seed uses the same bounded prefix of 256 labels, without independent source-semantic judgments. Historical T2 train/selection family cross-entropy equals T0 because those learned classifier tables are empty. The earlier cosine 1/MSE 0 reconstruction used the supplied target and is excluded from the present evaluation.

B Packed training equations

The packed reconstruction term averages over embedding coordinates,

$$L_{\text{rec}} = \frac{1}{n} \sum_i \frac{1}{d} \|\hat{e}_i - e_i\|_2^2, \quad (11)$$

and masked family/view cross-entropy is globally normalized over eligible rows M ,

$$L_{\text{CE}} = -\frac{1}{n} \sum_{i \in M} \sum_j q_{ij} \log \text{softmax}(\ell_i)_j. \quad (12)$$

The inspected update is SGD-style,

$$\Theta \leftarrow \Theta - \eta \text{Clip}(\nabla_{\Theta} L). \quad (13)$$

These identities describe the audited packed-CPU path. They are not a claim that every configured head was enabled in a deployed checkpoint, and they are not held-out source fidelity.

C Conditional proof-transfer proposition

Proposition 1 (conditional semantic consequence transfer). Assume a translator τ , bridge assumptions B , and a relation R_B that is nonvacuous on the admitted source models of Γ . If Equations 6–7 hold, premises are consistent in the relevant sense, and a sound target checker establishes

Table 5: Measured execution costs. Concurrent per-record intervals are sums, not end-to-end elapsed time or CPU time. Model request intervals exclude model loading. These scopes must not be added to enclosing job intervals.

Stage	Seconds	Scope
Native B, complete cohort	7332.8	sum of record intervals
Native C, complete cohort	13598.6	sum of record intervals
Repaired C, complete cohort	14100.6	sum of record intervals
Leanstrat, 64 proof calls	259.9	inference requests
Direct model A, all 100 units	2953.4	sum of record intervals
A, 97 returned responses	2601.9	returned API response intervals
Learned check-order replay	50.3	whole replay job
Full-document MiniLM encoding	1700.4	whole encoder job
Projection worker 1, four arms	3769.9	worker including preparation
Projection worker 2, four arms	3839.3	worker including preparation

454 $\tau(\Gamma) \models \tau(\phi)$, then $\Gamma, B \models_{L_s} \phi$ on the admitted source models. This is a paper-level conditional
 455 argument, not a theorem that every repository translator satisfies. Related-model existence, premise
 456 preservation, goal reflection, and checker identity must be discharged per supported fragment. The
 457 legacy TDFOL converter does not satisfy the contract.

458 **Proof.** Let s be an arbitrary admitted source model satisfying Γ and B . Related-model existence
 459 supplies a target model t with $R_B(s, t)$. Premise preservation gives $t \models \tau(\Gamma)$. The sound target
 460 consequence yields $t \models \tau(\phi)$, and goal reflection gives $s \models \phi$. Since s was arbitrary, $\Gamma, B \models_{L_s} \phi$
 461 over that admitted class. The bridge assumptions may be dropped only when they hold throughout
 462 the declared source profile.

463 D Empirical execution and reproducibility

464 The 97 returned natural-source responses account for 98,085 prompt and 30,415 completion tokens;
 465 3 failed requests have no returned token-usage report. Their observed failure wall time is included
 466 in the all-record total. The 64 proof calls use 9,470 and 6,056, respectively. Five separate serving-
 467 qualification calls use 11.218 request seconds and are excluded from those denominators. A failed
 468 chat-template qualification consumed 295.152 seconds of model loading and no model requests.
 469 The corrected proof job loaded in 281.246 seconds. The natural-source job loaded in 275.883 sec-
 470 onds, then kept that model warm across client batches under a 10,000-second wall limit. Historical
 471 launch arguments show an earlier 900-second envelope; retained runtime-override records identify
 472 the superseding limit without relabeling the original log.

473 The host is an NVIDIA GB10 with 20 logical CPU threads and approximately 128 GB of unified
 474 memory. Compiler jobs use bounded worker pools; each worker has a 4 GiB virtual-address limit.
 475 The Python runtime is 3.12.3, spaCy 3.8.14 with en_core_web_sm 3.8.0, and NumPy 1.26.4. Exact
 476 runtime, model, source, code, and observation hashes are retained in the supplement. Full-document
 477 encoding covers 19,955 windows and 4,819,558 tokens, using 1656.6 process-CPU seconds. The
 478 two projection workers use 6428.9 process-CPU seconds in total, including all four independently
 479 computed arms and shared source-feature preparation. Concurrent worker wall intervals are not
 480 summed as end-to-end elapsed time. The anonymous supplement contains metric-level per-unit ob-
 481 servations, fixed protocols, source/model identities, native proof receipts, and table-generation code.
 482 It excludes large model weights and private host paths. Complete local evidence retains raw requests,
 483 responses, compiler outputs, and exact state identities. Software qualification used constructed pos-
 484 itive and negative cases and the selection set. Infrastructure amendments and failed attempts are
 485 retained; no final-source outcome was used to select a revised model or compiler candidate.

486 D.1 Coverage of the original experimental programme

487 The original A–E comparison proposed four outcomes for every pipeline: coverage, independent
 488 source fidelity, useful checked proof transfer, and cost. Table 6 accounts for these 20 outcomes.
 489 The current experiments supply native representation, reconstruction, projection, scheduling, and

constructed proof measurements. They do not constitute execution of every proposed natural-source pipeline. In particular, a supported constructed proof does not establish useful transfer from an arbitrary legal source.

Table 6: Crosswalk for the original five-pipeline, four-outcome programme. Entries describe the actual measured populations; “not measured” is not a zero score.

Arm	Coverage / abstention	Independent fidelity	Useful native proof / transfer	Cost / latency
A	Measured on fixed 100-source subset	Not measured	Natural-source proof transfer not measured	Measured for 100 attempts
B	Measured on 1,913 sources	Not measured	Native IR cycle measured; useful proof transfer not measured	Measured for 1,913 attempts
C	Measured on 1,913 sources	Not measured	Source-withheld IR cycle measured; useful proof transfer not measured	Measured for 1,913 attempts
D	Full natural-source checked-bridge pipeline not executed	Not measured	Native assistance measured separately on 64 constructed goals	Constructed assistance costs measured; full natural-source cost not measured
E	Full learned source-to-proof pipeline not executed; check-order consumer measured	Not measured	Required structural checks replayed; natural-source proof transfer not measured	Check-order replay cost measured; full pipeline cost not measured

The policy–code–trace example is a finite semantic illustration, not a native source-code extraction benchmark. Historical supporting retrieval diagnostics used hashed trigram features and constructed judgments; they do not establish MiniLM/FAISS retrieval effectiveness or independently judged relevance. General retrieval-assisted premise admission, learned canonical-semantic mutation, and cross-domain natural-source proof transfer remain open experiments. The present measured check-order consumer is distinct from those broader mechanisms.

E Models, tools, and research assets

E.1 Executed model assistance

The proof and direct-formalization experiments use Leanstral-1.5-119B-A6B, with a locally cached NVFP4 GGUF quantization [Mistral AI, 2026, Frosty40, 2026]. The file contains 67,135,119,264 bytes. Its tokenizer requires the v15 message format; the GGUF does not supply a chat template, so we provide and hash the template explicitly. The natural-source model stayed loaded across client batches under a 10,000-second wall limit and stopped after all 100 attempts completed. The earlier proof jobs used 900-second wall envelopes. Serving qualification checks the rendered instruction and reasoning-setting tokens and a single beginning-of-sequence token before inference. We use reasoning effort none, temperature zero, fixed seeds, one request per item, and the token and wall budgets in the main text. This evaluates a bounded direct-proposal interface rather than the model’s published agentic configuration. Prompt text, responses, finish reasons, token counts, and native proof decisions are retained in the evidence. The natural-source prompt includes the registered operator vocabulary and an exact JSON schema. It never includes a compiler candidate or a semantic reference answer. All 100 natural prompts passed tokenizer-level context budget checks before the first request.

Native proof checking uses Lean 4.33.1 with fixed declarations. Z3 4.15.4 and CVC5 1.3.3 provide candidate first-order dispositions; native Lean reconstruction remains mandatory. The learned check-order consumer uses separate schema and provenance checks, not a proof kernel. Software agents independently inspected and qualified code boundaries; that engineering review is not human annotation of legal meaning.

One earlier constructed development call used the served model grok-4.6 through the xAI API, with 821 prompt and 54 completion tokens. A Codex gpt-5.6-terra fallback at high reasoning effort was configured but not invoked for that call. It is excluded from all current natural-source and 64-goal denominators. Agent assistance was also used for implementation, experiment orchestration, code inspection, manuscript editing, and citation checks. Authors remain responsible for the claims.

E.2 Embedding model and learned states

The complete-document source encoder is sentence-transformers/all-MiniLM-L6-v2 [Sentence Transformers, 2026], revision 1110a243fdf4706b3f48f1d95db1a4f5529b4d41, loaded locally with remote code disabled. Its 90,868,376-byte safetensors file has SHA-256 53aa51172d142c89d9012cce15ae4d6cc0ca6895895114379cacb4fab128d9db. We tokenize without truncation, divide content into 254-token windows, apply attention-mask mean pooling and L2 normalization per window, and form a content-token-weighted document average with final L2 normalization. Inference uses float32 on two CPU threads. All 15 selection vectors match the retained training-era encoder outputs exactly.

The adaptive states are distinct from the pretrained MiniLM encoder. We recover three memory-only states, three shared-parameter states, and three matched proof-head states. Extraction verifies each original state hash; large numerical arrays are preserved as exact float64 values. Prediction uses a fixed training-derived view alphabet, deontic.ir and knowledge_graphs.neo4j_compat. The modality and view classifiers have empty learned tables. Reconstruction bypasses the legacy target-guided helper. T3’s protected reconstruction state is identical to its paired T2 state; only the separate proof-head path is evaluated anew.

E.3 Assets and reproduction scope

The legal cohort comes from justicedao/patent-legal-corpus, revision 86c77cb650d30ee366983d1b2e25cd85e8c22e34. Upstream records declare reviewed, redistributable U.S. government works, and the dataset card declares CC0-1.0. We retain those declarations without asserting a new legal determination. The pinned MiniLM, upstream Leantral, and NVFP4 model cards each declare Apache-2.0. No code-licensing risk cohort is included in the natural-source results. The supplement identifies public model versions, source-unit hashes, normalization, aliases, and component membership.

The anonymous ZIP contains per-unit metric observations and scripts that regenerate reported counts, intervals, and tables, together with fixed protocols and proof-checking evidence. Multi-gigabyte learned states and the full external runtime are not shipped in that package. Thus table reproduction from measured observations is supported; a complete fresh inference run additionally needs the identified states and runtime. Their hashes identify exact inputs but are not an anonymous download service.

E.4 Broader impacts

Structured formalization can make missing approvals, incomplete audit conditions, or inconsistent premises easier to inspect. Its principal risk is misplaced authority: a structurally valid formula or accepted proof can be presented as a correct interpretation of a legal or security requirement. We report the scope of each check, preserve abstention and failure, and make no independent source-fidelity claim. Such tools should support inspection of explicit system requirements rather than substitute for accountable interpretation or justify monitoring beyond the declared task.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction describe executed native compilation, source-conditioned reconstruction, learned check ordering, and constructed proof assistance, with explicit limits on source fidelity and generalization.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The limitations section discusses unmeasured independent semantics, source prefixes, corpus dependence, empty classifier tables, the absence of a compression bottleneck, and the restricted proof suite.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The conditional transfer proposition states bridge assumptions, related-model existence, premise preservation, goal reflection, and soundness, with a complete argument in Appendix C.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Protocols, seeds, models, inputs, and metric definitions are disclosed. The supplement reproduces tables from per-unit measurements; a complete fresh inference run additionally requires the identified large local learned states, which are not included in the ZIP.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The supplement releases observations and table-generation code. The multi-gigabyte learned states and complete external runtime are identified but not distributed here, so complete open-access end-to-end reproduction is not yet provided.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The experimental-design section and supplement specify splits, three training seeds, native compilation budgets, fixed prompts, representation losses, and which stored heads were actually learned.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We report paired component-bootstrap intervals, define resampling units and seeds, and discuss the dominant component. Proof-study intervals use semantic families; no independence of all template variants is assumed.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The execution appendix reports current CPU/model limits and measured costs, plus earlier training costs and failures. Concurrent elapsed times are not presented as controlled throughput comparisons.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work uses public legal-source material and automated constructed tests; limitations and broader impacts distinguish machine checks from legal authority.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Potential benefits and misuse of automated legal or security formalization are discussed in Appendix E.4, including misleading claims of source-faithful verification.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No new large generative model or personal-data collection is released; this question does not apply to a high-risk model or dataset release.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Appendix E.3 identifies the legal corpus and upstream license declarations, MiniLM, and the LLaMA model and quantization provenance.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The supplement describes its metric observations, scripts, input identities, checksums, and omitted large assets. It includes an explicit distinction between table reproduction and fresh inference.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human-subject experiment was conducted, and no human agreement or semantic-fidelity score is reported.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human-subject research was performed.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Appendix E identifies the actual LLaMA model, prompts and inference budgets, MiniLM teacher, historical development use, and agent assistance with implementation and writing.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.